

The background of the image features a dark blue gradient on the left, transitioning into a large, vibrant, abstract shape on the right. This shape is composed of overlapping curved segments in shades of orange, pink, and purple, creating a dynamic, modern aesthetic.

AWS re:Invent

NOV. 27 – DEC. 1, 2023 | LAS VEGAS, NV

NET 325

Understanding false positives in your application security

Sagar Rao

(he/him)

Technical Account Manager,
Amazon Web Services

Girish Keshav

(he/him)

Solutions Architect,
Amazon Web Services



© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Agenda

- The problem and impact of false positives
- AWS Firewall Manager and AWS WAF overview
- Observability options and handling false positives
- Scenarios to troubleshoot false positives
- Q&A

The problem

- As internet users, how many of us have seen this before?
- But what if a genuine user sees this error?
- Termed as 'false positive' if a legitimate user is blocked erroneously by AWS WAF
- How do you deal with this at scale?



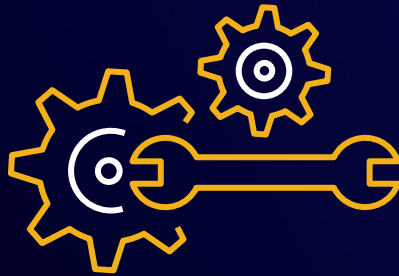
Impact of false positives

- Customer attrition
- Revenue loss
- Affects brand reputation



Customer-facing applications require security

FINANCIALLY IMPORTANT YET INCREASINGLY COMPLEX



The right tooling



Security expertise

Organizations must maintain up-to-date web traffic protection

AWS Edge security services



Amazon CloudFront



AWS WAF

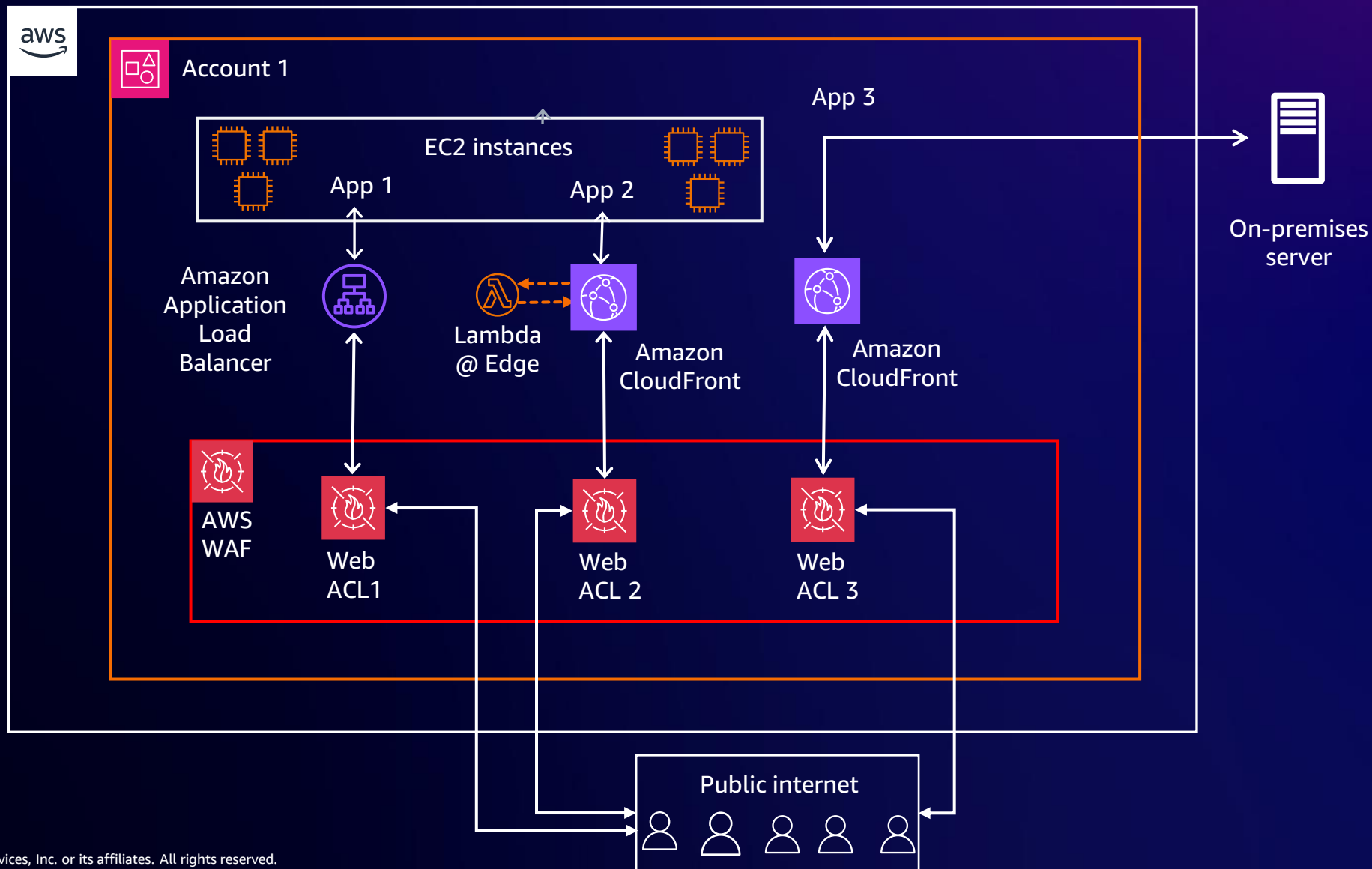


AWS Shield



AWS Firewall
Manager

AWS WAF deployment



Modern infrastructure needs to scale

Large number of accounts and resources



- Managing security policies centrally across all accounts and resources is a challenge

New applications created all the time



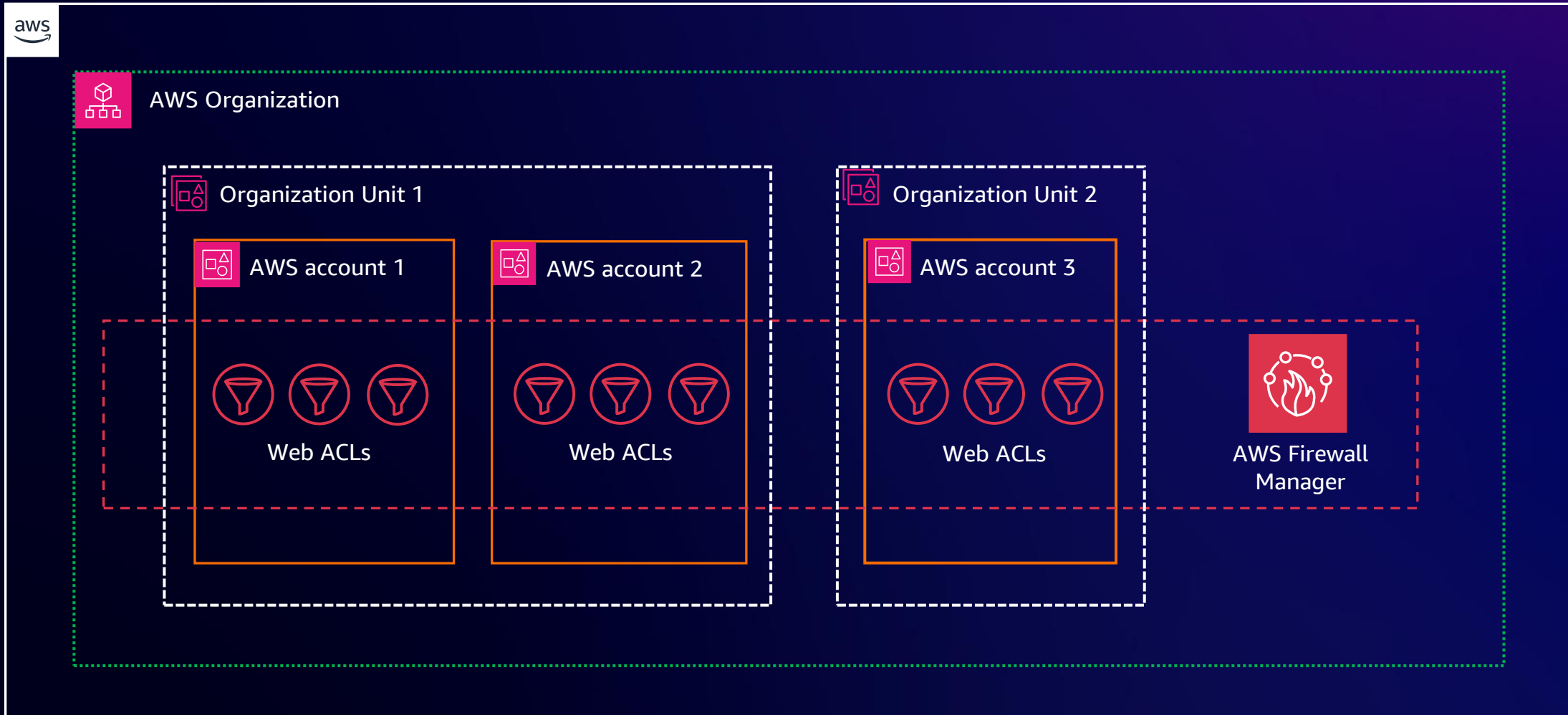
- Difficult to ensure that all applications are consistently protected from day 1

Central organization-wide visibility into threats



- No single place to monitor and respond to any threats across the organization

How to scale?



Observability options

CloudWatch metrics

- Metrics on every rule
- Allowed | Blocked | Counted | Challenge | CAPTCHA

Use case

Set alarms for notifications

Sampled web requests

- Detailed logs of a **sample** of requests
- Automatically available for every rule

Use case

Quickly test AWS WAF rules
Easy triaging on the console

Full logs

- Optionally enabled, detailed logs of **every** request
- Use Amazon Athena or Amazon CloudWatch Insights to query logs
- Log filtering options

Use case

Security analytics, monitoring, automation, auditing, and compliance

AWS WAF lifecycle



Example query to get a list of COUNT triggers

```
SELECT count(*) AS count, httpsourceid, httprequest.clientip, t.rulegroupid, t.nonTerminatingMatchingRules
FROM waf_logs
CROSS JOIN UNNEST(rulegrouplist) AS t(t)
WHERE action <> 'COUNT' AND cardinality(t.nonTerminatingMatchingRules) > 0
GROUP BY t.nonTerminatingMatchingRules, action, httpsourceid, httprequest.clientip, t.rulegroupid
ORDER BY "count" DESC
```

Mitigating false positives with the help of labels

- Labels allow you to create custom rules with more flexibility
- Amazon Managed Rules generate labels
- Create exception rules using labels to avoid false positives

Mitigating false positives with the help of labels

- Labels allow you to create custom rules with more flexibility
- Amazon Managed Rules generate labels
- Create exception rules using labels to avoid false positives

1

SizeRestrictions_Cookie_HEADER	☐ Count
SizeRestrictions_BODY	☒ Count
SizeRestrictions_URI_PATH	☐ Count

Mitigating false positives with the help of labels

- Labels allow you to create custom rules with more flexibility
- Amazon Managed Rules generate labels
- Create exception rules using labels to avoid false positives

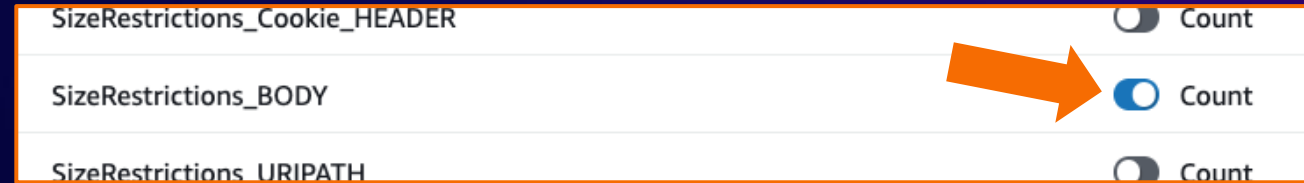
1

SizeRestrictions_Cookie_HEADER	☐ Count
SizeRestrictions_BODY	☒ Count
SizeRestrictions_URI_PATH	☐ Count

Mitigating false positives with the help of labels

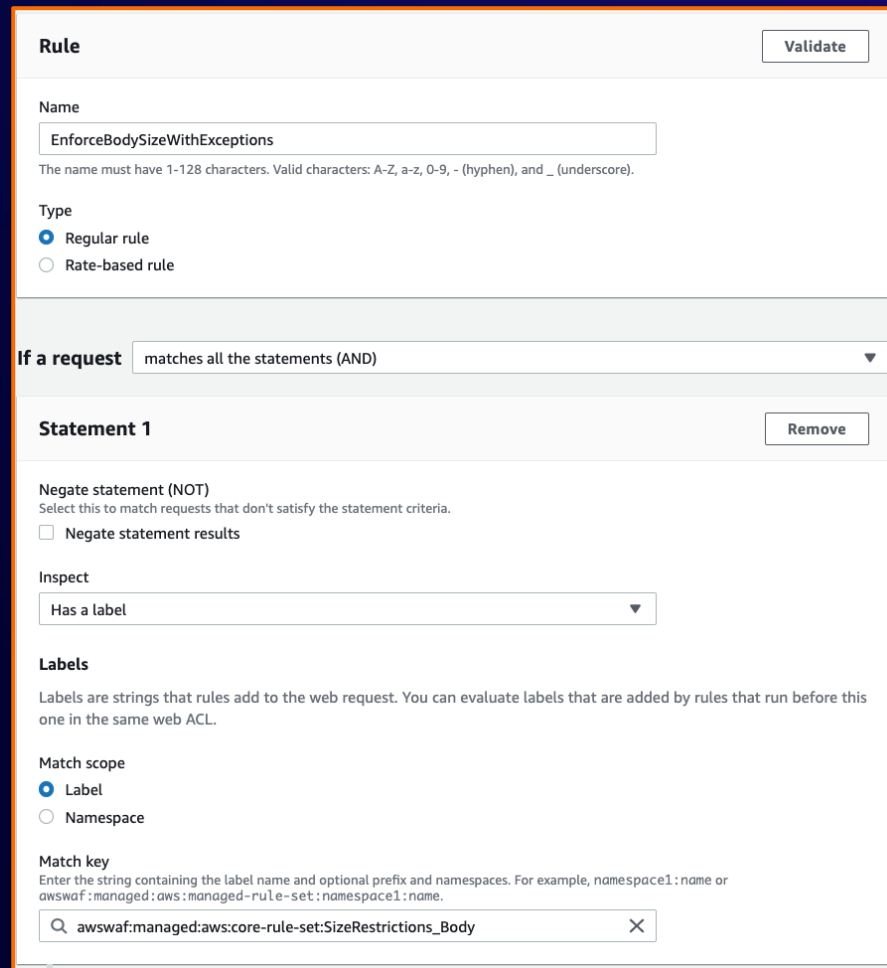
- Labels allow you to create custom rules with more flexibility
- Amazon Managed Rules generate labels
- Create exception rules using labels to avoid false positives

1



Rule Name	Count
SizeRestrictions_Cookie_HEADER	☐
SizeRestrictions_BODY	☒
SizeRestrictions_URI_PATH	☐

2



Rule Validate

Name

The name must have 1-128 characters. Valid characters: A-Z, a-z, 0-9, - (hyphen), and _ (underscore).

Type
☒ Regular rule
☐ Rate-based rule

If a request matches all the statements (AND)

Statement 1 Remove

Negate statement (NOT)
Select this to match requests that don't satisfy the statement criteria.
☐ Negate statement results

Inspect

Labels
Labels are strings that rules add to the web request. You can evaluate labels that are added by rules that run before this one in the same web ACL.

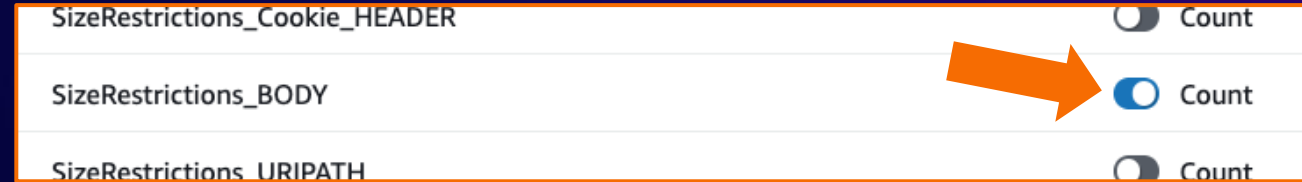
Match scope
☒ Label
☐ Namespace

Match key
Enter the string containing the label name and optional prefix and namespaces. For example, namespace1:name or aws:managed:aws:managed-rule-set:namespace1:name.

Mitigating false positives with the help of labels

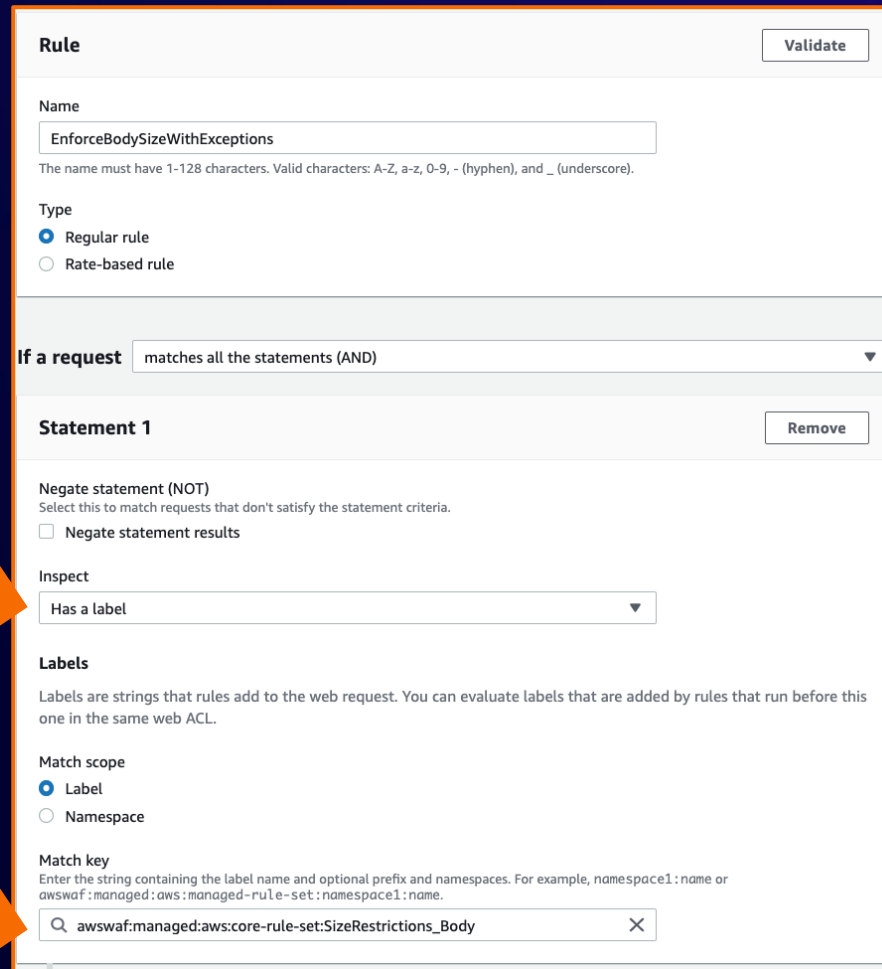
- Labels allow you to create custom rules with more flexibility
- Amazon Managed Rules generate labels
- Create exception rules using labels to avoid false positives

1



Rule Name	Count
SizeRestrictions_Cookie_HEADER	☐
SizeRestrictions_BODY	☒
SizeRestrictions_URI_PATH	☐

2



Rule Validate

Name

The name must have 1-128 characters. Valid characters: A-Z, a-z, 0-9, - (hyphen), and _ (underscore).

Type
☒ Regular rule
☐ Rate-based rule

If a request matches all the statements (AND)

Statement 1 Remove

Negate statement (NOT)
Select this to match requests that don't satisfy the statement criteria.
☐ Negate statement results

Inspect

Labels
Labels are strings that rules add to the web request. You can evaluate labels that are added by rules that run before this one in the same web ACL.

Match scope
☒ Label
☐ Namespace

Match key
Enter the string containing the label name and optional prefix and namespaces. For example, namespace1:name or aws:managed:aws:core-rule-set:namespace1:name.

Mitigating false positives with the help of labels

- Labels allow you to create custom rules with more flexibility
- Amazon Managed Rules generate labels
- Create exception rules using labels to avoid false positives

1

SizeRestrictions_Cookie_HEADER ☐ Count

SizeRestrictions_BODY ☒ Count

SizeRestrictions_URI_PATH ☐ Count

2

Rule Validate

Name
EnforceBodySizeWithExceptions
The name must have 1-128 characters. /valid characters:

Type
☒ Regular rule
☐ Rate-based rule

If a request matches all the statements (AND)

Statement 1

Negate statement (NOT)
Select this to match requests that don't satisfy the statement criteria.
☐ Negate statement results

Inspect
Has a label

Labels
Labels are strings that rules add to the web request one in the same web ACL.

Match scope
☒ Label
☐ Namespace

Match key
Enter the string containing the label name and optional namespace:
aws:managed-aws:managed-rule-set:namespace

3

AND

NOT Statement 2 Remove

Negate statement (NOT)
Select this to match requests that don't satisfy the statement criteria.
☒ Negate statement results

Inspect
Single query parameter

Query argument
PostData

Match type
Exactly matches string

String to match
True

Text transformation
AWS WAF applies all transformations to the request before evaluating it. If multiple text transformations are added, then text transformations are applied in the order presented below with the top of the list being applied first.
None

Add text transformation
At least one text transformation is required. You can add up to 10 text transformations.

aws:managed-aws:core-rule-set:SizeRestrictions_Body

Mitigating false positives with the help of labels

- Labels allow you to create custom rules with more flexibility
- Amazon Managed Rules generate labels
- Create exception rules using labels to avoid false positives

1

SizeRestrictions_Cookie_HEADER ☐ Count

SizeRestrictions_BODY ☒ Count

SizeRestrictions_URI_PATH ☐ Count

2

Rule

Name
EnforceBodySizeWithExceptions
The name must have 1-128 characters. /valid characters:

Type
☒ Regular rule
☐ Rate-based rule

If a request matches all the statements (AND)

Statement 1

Negate statement (NOT)
Select this to match requests that don't satisfy the statement criteria.
☐ Negate statement results

Inspect
Has a label

Labels
Labels are strings that rules add to the web request in the same web ACL.

Match scope
☒ Label
☐ Namespace

Match key
Enter the string containing the label name and optional namespace.
aws:managed:aws:core-rule-set:SizeRestrictions_Body

3

AND

NOT Statement 2

Negate statement (NOT)
Select this to match requests that don't satisfy the statement criteria.
☒ Negate statement results

Inspect
Single query parameter

Query argument
PostData

Match type
Exactly matches string

String to match
True

Text transformation
AWS WAF applies all transformations to the request before evaluating it. If multiple text transformations are added, then text transformations are applied in the order presented below with the top of the list being applied first.
None

Add text transformation

At least one text transformation is required. You can add up to 10 text transformations.

Mitigating false positives with the help of labels

- Labels allow you to create custom rules with more flexibility
- Amazon Managed Rules generate labels
- Create exception rules using labels to avoid false positives

The screenshot illustrates the configuration of a custom rule in the AWS WAF console. It is divided into several sections, with orange arrows and numbers highlighting key features:

- 1. Rule List:** A table at the top shows existing rules: `SizeRestrictions_Cookie_HEADER`, `SizeRestrictions_BODY`, and `SizeRestrictions_URI_PATH`. Each rule has a 'Count' toggle. An arrow points to the 'Count' toggle for `SizeRestrictions_BODY`, which is currently turned on.
- 2. Rule Configuration:** The main configuration area for the rule named `EnforceBodySizeWithExceptions`. It includes a 'Validate' button and a 'Type' selector set to 'Regular rule'.
- 3. Statement Configuration:** The 'Statement 1' section is configured with 'Negate statement (NOT)' checked, 'Inspect' set to 'Has a label', and 'Match scope' set to 'Label'. The 'Match key' field contains the label `aws:managed:aws:core-rule-set:SizeRestrictions_BODY`.
- 4. Action Configuration:** The 'Action' section is configured with 'Block' selected. The 'Text transformation' dropdown is set to 'None'.
- 5. Logic Operator:** An arrow points to the 'AND' operator button, indicating the logical relationship between statements.
- 6. Negate Statement:** An arrow points to the 'Negate statement (NOT)' section, which is checked to match requests that do not satisfy the criteria.

Mitigating false positives with the help of labels

- Labels allow you to create custom rules with more flexibility
- Amazon Managed Rules generate labels
- Create exception rules using labels to avoid false positives

SizeRestrictions_Cookie_HEADER	☐	Count
SizeRestrictions_BODY	☒	Count
SizeRestrictions_URI_PATH	☐	Count

Rule

Name: EnforceBodySizeWithExceptions

Type: ☒ Regular rule

If a request matches all the statements (AND)

Statement 1

Negate statement (NOT)

Inspect: Has a label

Labels

Match scope: ☒ Label

Match key: aws:managed:aws:core-rule-set:SizeRestrictions_Body

NOT Statement 2

Negate statement (NOT)

Inspect: Single query parameter

Query argument: PostData

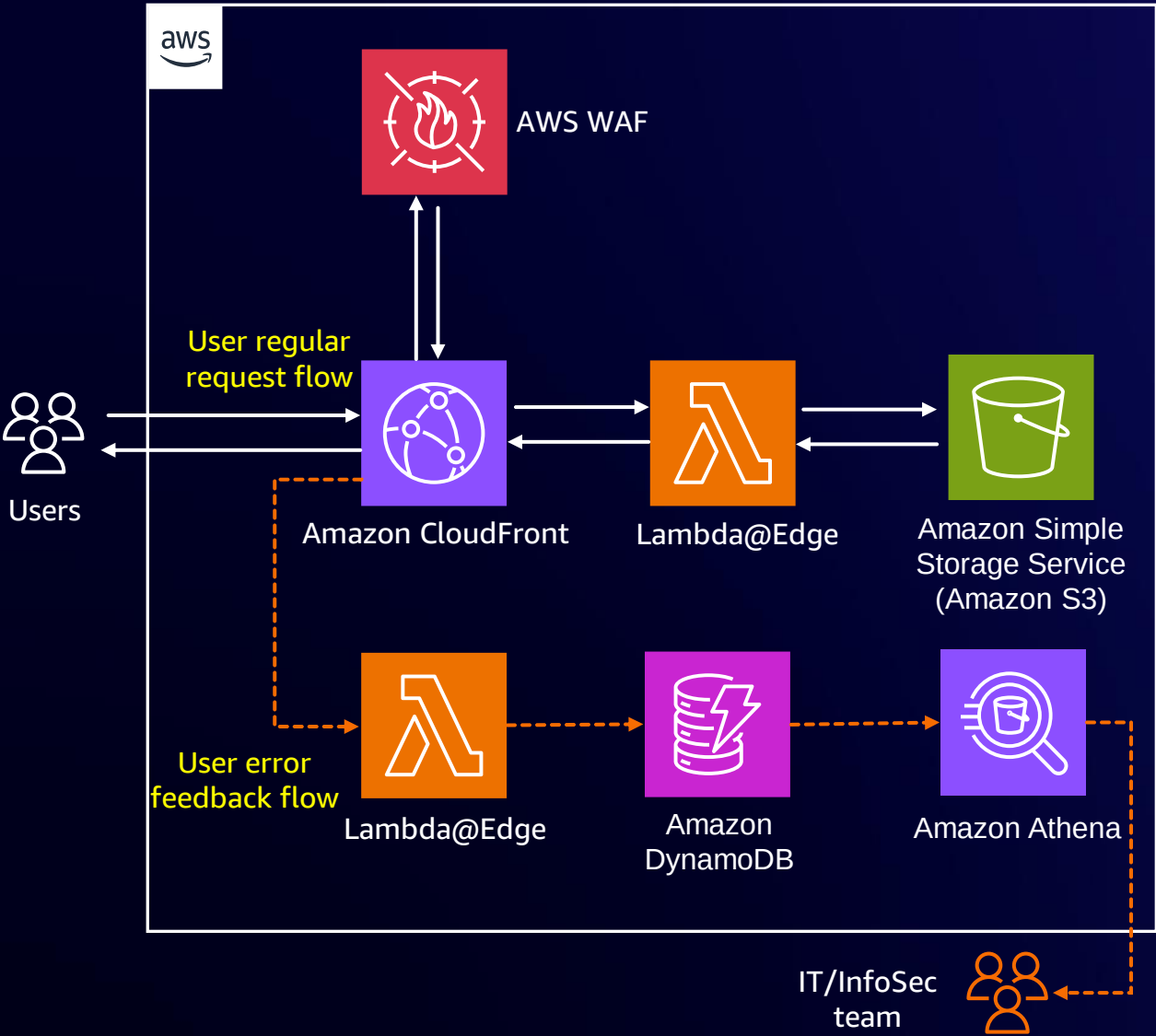
Match type: Exactly matches string

String to match: True

Text transformation: None

Action: ☒ Block

Handling false positive errors



403 Forbidden

Report if you think you came here by mistake...

CloudFront request id: w3PbPZF_X0Qw_LQZAA0cmwEXd8vx2YuZcorGnNJJfg11-iBKvi9uKQ==

Click to report

False positives scenarios

Scenario 1

- Legitimate users reporting that they are getting a 403 on your website
- Your developer team made a code change to an application the previous day
- Security operations center reports multiple alerts of 403 errors on the application

Can you think of a way to solve these false positives?



First step – Analyze logs based on the alarm

If you have set up a CloudWatch alarm for Blocked Request, you will get a notification once the threshold is met

Alarm Details:

- Name: WAF_SQLi_Blocked_Requests
- Description: This is an alarm for requests blocked because of SQLi
- State Change: **OK -> ALARM**
- Reason: Threshold Crossed: 1 out of the last 1 datapoints [7.0 (06/10/23 01:07:00)] was greater than or equal to the threshold (5.0)
- Timestamp: Friday 06 October, 2023 01:22:54 UTC
- AWS Account: **772753556250**
- Alarm Arn: arn:aws:cloudwatch:us-east-1:772753556250:alarm:WAF_SQLi_Blocked_Requests

Monitored Metric:

- MetricNamespace: AWS/WAFV2
- MetricName: BlockedRequests
- Dimensions: **[WebACL = FMManagedWebACLV2-AWS_FMS_WAF_Rule-1628096141768] [Rule = AWS-AWSManagedRulesSQLiRuleSet]**
- Period: 900 seconds
- Statistic: Sum

Example query to analyze logs

After the alarm is triggered, query Athena logs based on Count Action

```
SELECT
  COUNT(*) AS count,
  webaclid,
  terminatingruleid,
  terminatingrulematchdetails,
  httprequest.clientip,
  httprequest.uri
FROM "awswafsecurityautomations_oqnvln"."waf_logs4"
WHERE action='BLOCK' AND terminatingruleid='AWS-AWSManagedRulesSQLiRuleSet' AND from_unixtime(timestamp/1000) > now()
- interval '1' day
GROUP BY webaclid, terminatingruleid, terminatingrulematchdetails, httprequest.clientip, httprequest.uri
ORDER BY count DESC
```

Analyzing in action (video)

Resource Groups & Tag Editor

Amazon Athena > Query editor

Editor | Recent queries | Saved queries | Settings

Workgroup: primary

Athena now supports typeahead code suggestions to speed up SQL query development
Typeahead suggestions are turned on by default. You can change this setting in query editor preferences. [Edit preferences](#)

Data

Data source: AwsDataCatalog

Database: awswafsecurityautomations_oqnvln

Tables and views [Create](#)

Filter tables and views

▼ **Tables (4)**

- + app_access_logs Partitioned
- + waf_logs
- + waf_logs3
- + waf_logs4

► **Views (0)**

SQL Ln 1, Col 1

[Run](#) [Explain](#) [Cancel](#) [Clear](#) [Create](#)

☐ Reuse query results up to 60 minutes ago

[Query results](#) [Query stats](#)

Results

[Copy](#) [Download results](#)

CloudShell Feedback

© 2023, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Second step – Fine-tune

Change SQLi cookie rule from Block to Count

SQLi_COOKIE
Rule action: **Block**

Override to Count ▲

Q |

Override to Allow

Override to Block

Override to Count

Override to CAPTCHA

Override to Challenge

↶ Remove Override

Add another rule after the Amazon Managed SQLi rule

Rules (6)

Q Find rules

☐	Name
☐	AWS-AWSManagedRulesSQLiRuleSet
☐	ExceptionSQLiCookie
☐	Bypasspostman
☐	RBR
☐	demosuspect
☐	demosuspectbypass

Block all SQLi requests except sessionId cookie using WAF SQLi labels

```
{
  "Name": "ExceptionSQLiCookie",
  "Priority": 1,
  "Statement": {
    "AndStatement": {
      "Statements": [
        {
          "LabelMatchStatement": {
            "Scope": "LABEL",
            "Key": "aws:waf:managed:aws:sql-database:SQLi_Cookie"
          }
        },
        {
          "NotStatement": {
            "Statement": {
              "SizeConstraintStatement": {
                "FieldToMatch": {
                  "Cookies": {
                    "MatchPattern": {
                      "IncludedCookies": [
                        "mysessionid",
                        "mynewsessionid"
                      ]
                    },
                    "MatchScope": "KEY",
                    "OverrideHandling": "CONTINUE"
                  }
                },
                "ComparisonOperator": "GE",
                "Size": 0,
                "TextTransformations": [
                  {
                    "Priority": 0,
                    "Type": "NONE"
                  }
                ]
              }
            }
          }
        }
      ]
    }
  }
}
```

Fine-tuning in action (video)

Resource Groups & Tag Editor

WAF & Shield

AWS WAF

Getting started

Web ACLs

Bot control dashboard

Application integration

New

IP sets

Regex pattern sets

Rule groups

AWS Marketplace managed rules

Switch to AWS WAF Classic

AWS Shield

Getting started

Overview

Protected resources

Events

Global threat dashboard

AWS Firewall Manager

Getting started

Security policies

PREFMManaged-AWSManagedRulesWindowsRuleSet-1696533634771

Action set by rule group

Enabled

Enabled

PREFMManaged-AWSManagedRulesWindowsRuleSet-1696533634771

New AWS managed rule groups available:

Account takeover prevention: Provides protection against unauthorized access to accounts on your site.

Account creation fraud prevention: Provides protection against the creation of fraudulent accounts on your site.

Add managed rule groups

Rules (5)

EditDeleteAdd rules

Find rules

	Name	Action	Priority	Custom response
☐	AWS-AWSManagedRulesSQLiRuleSet	Use rule actions	0	-
☐	Bypasspostman	Block	1	-
☐	RBR	Block	2	-
☐	demosuspect	Count	3	-
☐	demosuspectbypass	Block	4	-

Last rule groups, from AWS Firewall Manager (1)

These rule groups are managed by the AWS Firewall Manager admin. AWS WAF will evaluate them last, after evaluating any rules you have added to this web ACL.

Name	Action	Sampled requests	CloudWatch metrics	Metric name
POSTFManaged-AWSManagedRulesAmazonIpReputationList-1696533634770	Action set by rule group	Enabled	Enabled	POSTFManaged-AWSManagedRulesAmazonIpReputationList-1696533634770

CloudShellFeedback

© 2023, Amazon Web Services, Inc. or its affiliates. PrivacyTermsCookie preferences

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Third step – Retest and monitor (video)

The screenshot displays the AWS Athena console interface. At the top, the navigation bar includes the AWS logo, 'Services', a search bar, and the region 'N. Virginia'. The main header shows 'Resource Groups & Tag Editor'. A notification banner states: 'Athena now supports typeahead code suggestions. Typeahead suggestions are turned on by default.'

The left sidebar contains a 'Data' section with 'Data source' set to 'AwsDataCatalog' and 'Database' set to 'awswafsecurityautomations_oqnvln'. Below this is a 'Tables and views' section with a 'Create' button and a search bar. A list of tables is shown: 'app_access_logs' (Partitioned), 'waf_logs', 'waf_logs3', and 'waf_logs4'. There are also 'Views (0)'. The right sidebar has an 'Edit preferences' button and a 'Reuse query results' toggle set to 'up to 60 minutes ago'.

The main content area shows a query execution interface. A terminal window titled 'raosga — -zsh — 107x24' is overlaid, displaying the command: `(pyk2env) raosga@3c22fb21bedb ~ % curl -I -H "User-Agent:mybrowser5_version_1.5" d3any5cetxg2c1.cloudfront.net --cookie "mynewsessionid=null; a=1;"`. Below the terminal, the 'Query results' tab is active, showing a 'Completed' status with 'Time in queue: 155 ms', 'Run time: 827 ms', and 'Data scanned: 1.03 MB'. The 'Results (49)' section is visible at the bottom, with a search bar and pagination controls.

The footer includes the AWS logo, 'CloudShell', 'Feedback', and copyright information: '© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.' There are also links for 'Privacy', 'Terms', and 'Cookie preferences'.

Scenario 2

- You are concerned that false positives may be happening at scale across other applications
- You want a way to quickly analyze requests across different applications and take actions accordingly
- Your team has been asked to build visualizations for easier reporting and analysis

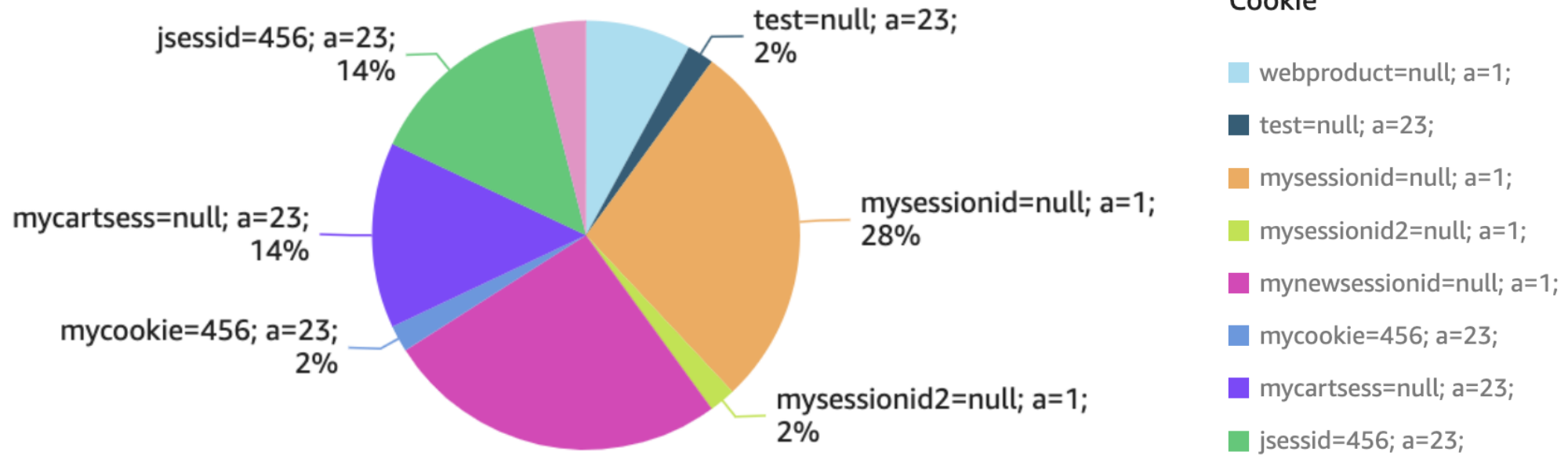
Can you think of a way to solve this?



First step - Analyze

- Build dashboards and visualizations based on the existing logs

Blocked Request based on Cookies



- Data flow for visualization



AWS WAF



Amazon S3



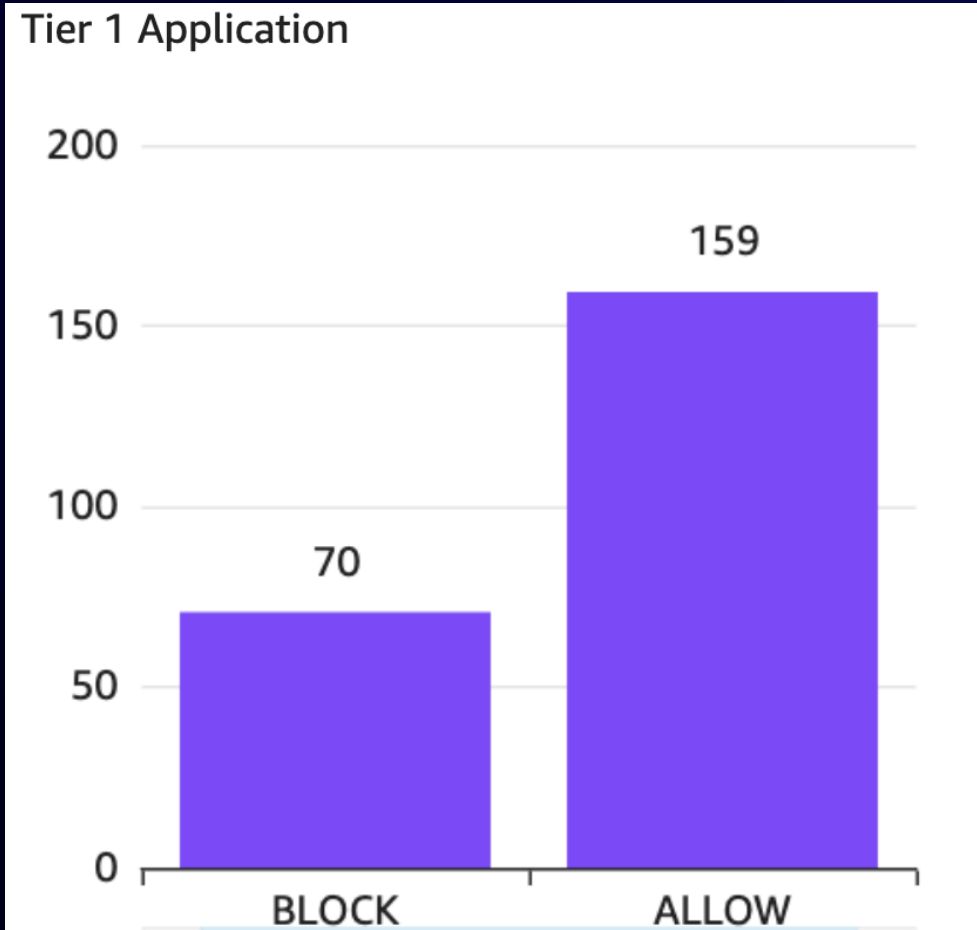
Amazon Athena



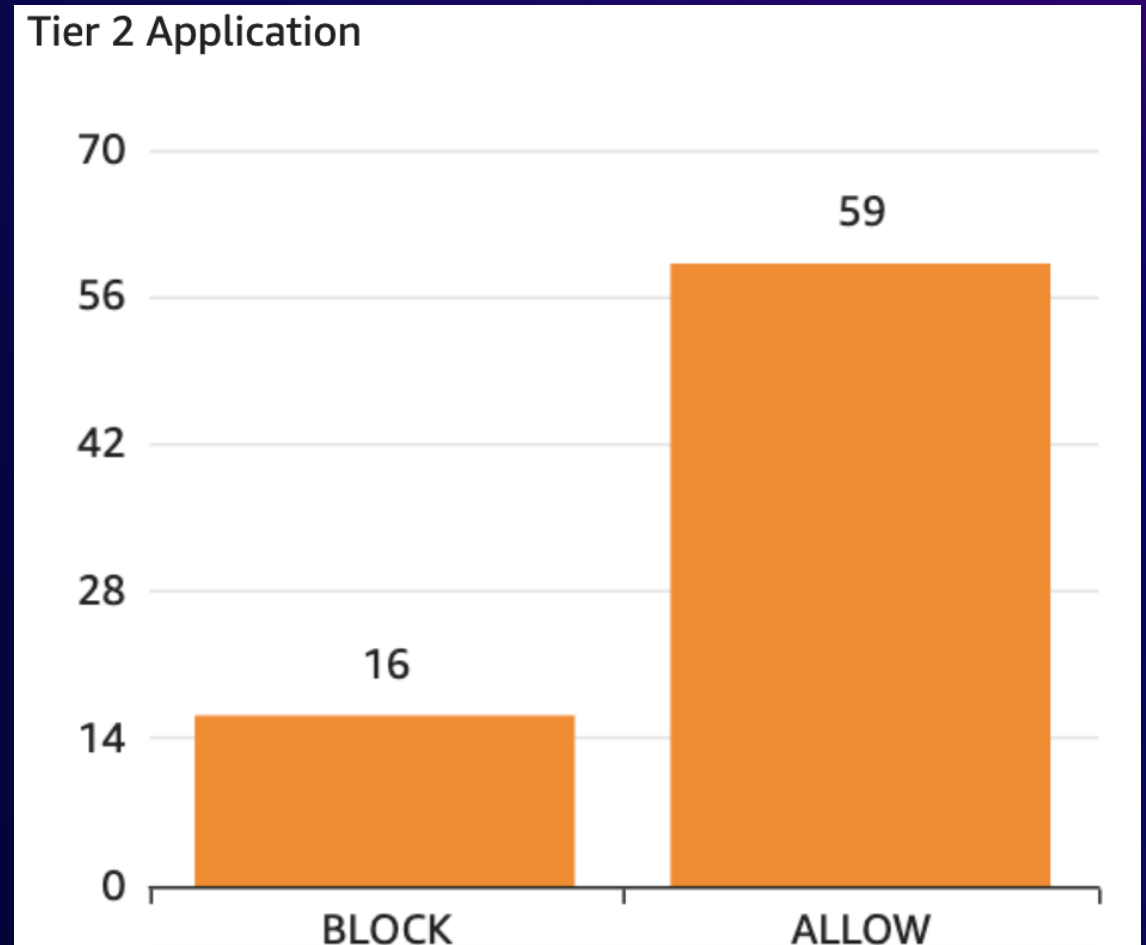
Amazon QuickSight

First step – Analyze

- Analyze based on different applications at scale

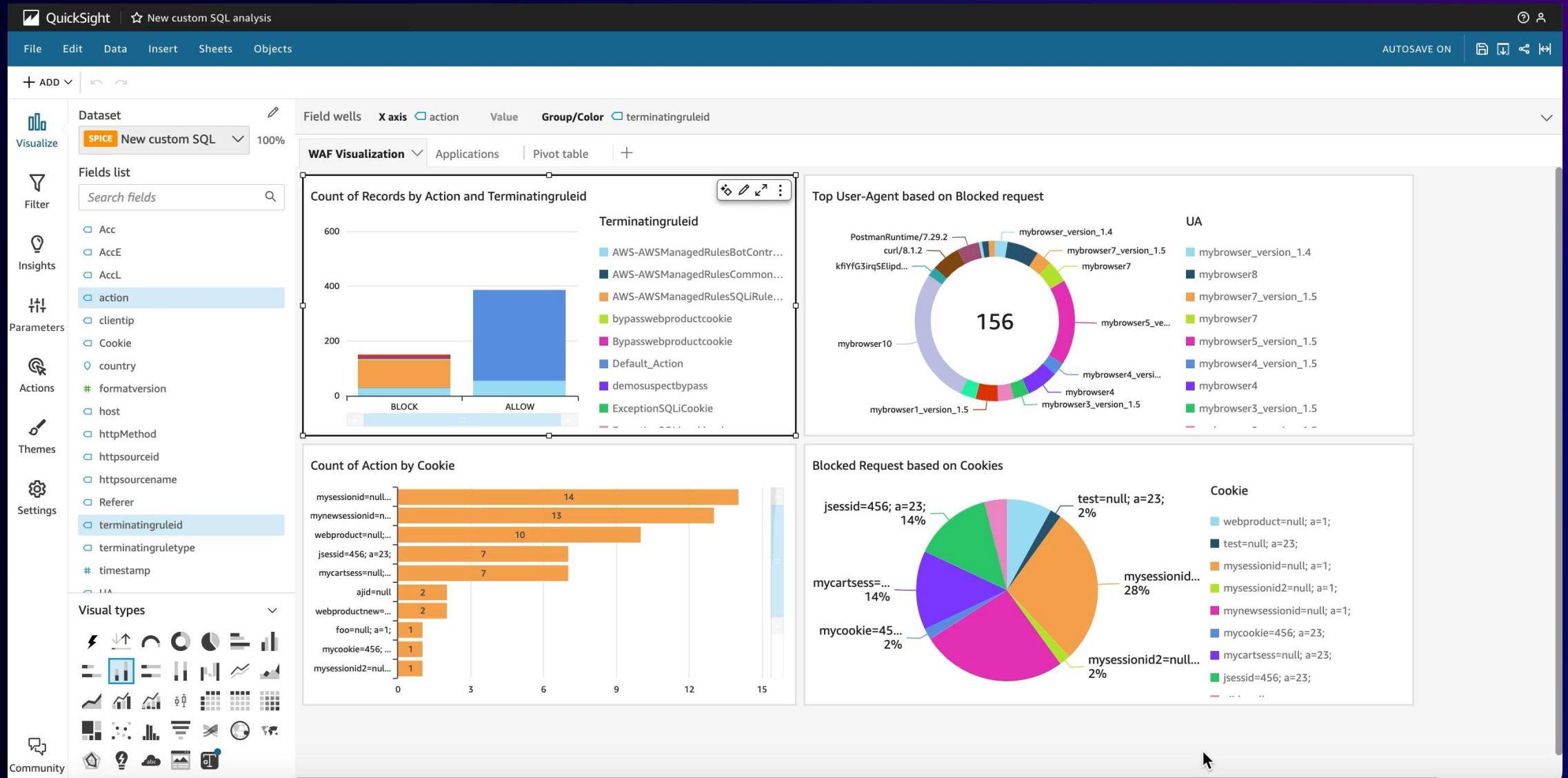


 d3any5cetxg2cl.cloudfront.net



 d3sp08yhnmg9e.cloudfront.net

Analyzing in action (video)



Fine-tuning

You have two options to fine-tune in this case:

- Follow the **same process mentioned in Scenario 1** and fine-tune the rules within the application web ACL

OR

- Make changes to the main firewall manager policy; add SQL injection and Exception rule to the Rule Groups
- You could also set anomaly detection on these visualizations to alert you on false positives

Summary

Deployment

- Deploy at scale using AWS Firewall Manager
 - Always start with COUNT mode
- Implement error reporting for end users

Monitoring

- Set up Amazon CloudWatch alarms
- Review logs using Amazon Athena or Amazon CloudWatch Insights
- Build visualizations for logs using Amazon QuickSight

Fine-tuning

- Fine-tune using AWS WAF labels
- Add exceptions after Amazon Managed Rules
- Fine-tune AWS Firewall Manager policies to achieve this at scale

Resources



AWS WAF



AWS Firewall Manager



Working with false
positives on AWS WAF



Working with AWS
Managed Rules blog



Querying AWS WAF logs
using Athena

Thank you!



Please complete the session survey in the mobile app

Sagar Rao

 [linkedin.com/in/sagarrao29](https://www.linkedin.com/in/sagarrao29)

Girish Keshav

 [linkedin.com/in/girishkeshav](https://www.linkedin.com/in/girishkeshav)